

Using the Lottery Ticket Hypothesis to Decrease the Size of LLMs

Author: [Len Washington III](#)  (A20484223)

Artifacts:  [lwashington3/cs595-2-project](#)



[lwashington3/lottery-ticket-hypothesis-for-llm](#)

Systemization/reproduction

1. Motivation

The introduction of transformers and large language models (LLMs) have greatly improved the efficiency of NLP tasks, allowing a single model to handle a multitude of tasks. One such example where LLMs have been implemented is the [EXSCLAIM](#) pipeline,¹ which utilizes LLMs to split captions from research papers to match with any subfigures within the overall (master) figure. However, the biggest problem with running EXSCLAIM on its current servers is that there are no GPUs, which significantly reduces how fast the pipeline can run. Coupled with the dozens of captions that may need to be separated, the Caption Distributor is easily the largest bottleneck within the pipeline. The default LLM (Llama 3.2) does a fair job of splitting these captions, though it will either need to be replaced by a better model (which would have to be larger) or improved in some way. To address these issues, a method was sought out to increase the inference speed without disrupting the current accuracy of the model. A pruning technique known as the Lottery Ticket Hypothesis was chosen, since it had claims of reducing a neural network's weights by 95% while still maintaining similar accuracies as the unpruned version.

2. Background

The Lottery Ticket Hypothesis states that “a randomly-initialized, dense neural network contains a subnetwork that is initialized such that—when trained in isolation—it can match the test accuracy of the original network after training for at most the same number of iterations.”² Essentially, by training a randomly-initialized neural network, a subnetwork (or winning ticket) can be discovered and retrained from the beginning, matching the accuracy of the original model.

The paper describes two methods towards pruning the network, one-shot pruning, and iterative pruning. For one-shot pruning, the trained network is pruned once, with $p\%$ of the weights being masked out (pruned) while the rest are reset to the initially-random values. There is also iterative pruning, which repeatedly trains, prunes, and resets the network over n rounds, each time pruning another $p\frac{1}{n}\%$. [Brix, Bahar, and Ney](#) describe further methods for utilizing the Lottery Ticket Hypothesis with transformers. One such method recommends applying the computed mask m to one of the earlier checkpoints from when the initially-randomized model was trained, rather than the initial model.

Since EXSCLAIM's pipeline was summarizing scientific captions into a list of strings,

a similar dataset was sought. As such, one dataset stood out, [MMInstruction/ArxivCap](#),⁴ which was designed for training parsers to understand Arxiv papers. The 755 GB dataset was filtered, retaining only what was necessary, before re-uploading it to HuggingFace. This dataset, [lwashton3/ArxivCap-Minimal](#), however, did not seem to work well with most LLMs, and was replaced for the duration of the project with [abisee/cnn_dailymail](#).

3. Methodology

There were several steps involved in implementing the Lottery Ticket Hypothesis when neither the initial model, nor the training dataset, were available. The first is to obtain the raw weights of a chosen LLM, as well as a dataset that would be used to fine-tune the model to help bridge any assumptions made by (2) having the initial training dataset for the pruning process. Once the model has been fine-tuned, each layer within the model will be subjected to one-shot magnitude pruning, where the weights are sorted by magnitude, then the smallest $p\%$ of parameters are masked and the model is re-trained.¹

To evaluate the results, a pipeline was constructed to run through the original (unpruned) model, as well as several pruned models using the held-out portion of the dataset. Once the models have been loaded into memory, the system evaluates each model against the given test row. The model is called once using HuggingFace’s native implementation with timers set to measure how long it takes for the first token to be generated, as well as the overall run time. After that, the model is called again, this time using PyTorch’s implementation, to receive the logits from the output which can be used to calculate the perplexity of the prediction. The last metric, the ROGUE-L F1 score, can be calculated with either the true test value or the predicted value of the baseline acting as the target.

4. Experimental Setup

Table 1: Experimental Specifications

GPU	NVIDIA A100 80GB PCIe
CPU	AMD EPYC 7763 64-Core Processor
Cores	256
RAM	503.414455 GB
Disk Space	418 GB
Chameleon Cloud Site	CHI@UC
Chameleon Cloud Node Name	gigaio-compute-04

¹Originally, the implementation missed the step of re-training the masked models. Those models and masks can be seen at <https://huggingface.co/lwashton3/lottery-ticket-hypothesis-masks>. The results they had were similar to the properly retrained versions found at <https://huggingface.co/lwashton3/lottery-ticket-hypothesis-models>.

Python 3.14.4 was the only language involved, and the software stack was mostly composed of the HuggingFace suite, namely Transformers, Datasets, and Safetensors (the library used to access the raw tensor parameters). The full list of libraries and versions can be seen in Appendix Section A. The baseline model that was used for pruning and evaluation was [google/gemma-3-1b-it](#) (Gemma 3) along with [abisee/cnn_dailymail](#) (CNN DailyMail) as the dataset. The metrics used to evaluate the effectiveness of pruning were time-to-first-token (TTFT), time per output tokens (TPOT), run time, ROGUE-L F1 score, and Perplexity.

5. Results

First, it should be noted that the original Gemma 3 model was not accurate when it came to summarizing the CNN DailyMail dataset. The low results led to substituting in Gemma’s responses for any metrics that required the true value (namely ROGUE-L) so the pruned models could be judged more accurately.

The results for the experiment turned out rather well, with the pruned models taking less time to run, while being able to produce the exact same response as the original model.² The first row of CNN DailyMail’s test set can be found in Tables 2-3, which show the models that were and were not pre-trained before pruning, respectively, and Figures 1-5. The only metric that was not better than the original was the perplexity, which shows that the pruned models are a lot less certain about their responses than the original model is. However, given that the other metrics were improved and the output was the same, this does not matter as much.

The results for testing pre-training the model beforehand can be seen in Figures 6-9. The results were comparable, though pre-trained models seemed to have the edge a bit more often.

6. Profiling Analysis

Originally, reading in the models for the evaluation pipeline would take ~ 40 seconds, which led to 50+ minute wait times when trying to load in 79 models into memory.³ The read time was later reduced to 23–28 seconds, reducing the wait time.⁴ The training functions were also improved by running a binary search to figure out how many rows the GPU could hold for a given dataset and model, without having to guess or constantly use a single row for training and validation.

²Note: this happens when the model has `max_new_tokens` set to 0. Anything more will result in extra tokens that do not make sense towards the output. Since the pipeline could not have `max_new_tokens` to anything less than 1, `max_new_tokens` was set to 1 and the last token for each output was ignored.

³These results were from the masked models that were not re-trained. Saving only the masks took considerably less memory, and allowed for more models to exist at once. The enhancements are still relevant for the corrected codebase.

⁴Memory profiles for the evaluation pipeline are available in the GitHub artifact.

7. Reproducibility

The artifact can be found at <https://github.com/lwashington3/cs595-2-project>. By cloning the repository, Docker Compose can be used to run the LTH module, along with containing scripts used to automatically generate the figures and tables in this paper. Running `docker compose run -it project` in the repo’s root directory will run the ablation script with default values to generate masks for pruned models (pre-trained and not pre-trained), as well as the figures and tables during the evaluation. Other scripts are available in the `scripts` folder, along with documentation for how to set your own variables. With default values, the models should appear in the `models` folder, which will then be sorted by process name (default is “Ablation {UNIX timestamp that the script started}”), training status (trained or untrained), then the pruning percentage used. Within the `evaluation` folder, a folder with the same process name as before will contain a folder for each row in the test dataset that was evaluated. Each of those folders will contain the figures (saved as PNG, SVG, and $\text{\LaTeX}$ compatible-PGF formats), as well as the tables pre-processed in a $\text{\LaTeX}$ file. There will be two copies of each file’s stem, one with `_baseline`, which has the baseline value added, such as Figures 1-5, and one only containing the pruned models, such as Figures 6-10.

8. Limitations

Currently, the pre-training process is limited by the size of the GPU. Without a GPU, the training could run for hours without visibly finishing the first example. Even then, a 32 GB Nvidia GeForce RTX 5090 couldn’t fit any rows alongside a few LLMs, such as [meta-llama/Llama-3.1-8B-Instruct](#)⁵; and the 80 GB GPU mentioned in Table 1 couldn’t handle training [meta-llama/Llama-3.1-8B-Instruct](#), [Qwen/Qwen3-4B-Instruct-2507](#), [Qwen/Qwen3.5-2B](#), and could only hold 6 rows alongside Gemma. The size of the hard drive was also a massive limiter, with 418 GB only holding 40 fully trained models, limiting the ability to store models to evaluate later on.

9. Future Works

There are several immediate steps that would have been taken if there was more time. The first would be to figure out how to have more control over the training process, since it seems that the current behavior is to load everything onto the GPU immediately, instead of loading and unloading rows as needed so the entire dataset would eventually be used. The second would be to implement the ability to choose between one-shot pruning and iterative pruning, since the current codebase can only run one-shot. The pipeline should also be updated to allow masking with multiple pruning parameters at once instead of having to restart the pipeline for each pruning percentage, since this unnecessarily offloads and onloads the same data. Tests should also be done to see if structurally pruning different parts of the model,

⁵Which resulted in Gemma 3 being the default model.

such as only the head or removing an entire layer, can positively reduce the size of the model without compromising accuracy.

10. References

- [1] Eric Schwenker et al. *EXSCLAIM! – An automated pipeline for the construction of labeled materials imaging datasets from literature*. 2021. arXiv: [2103.10631](https://arxiv.org/abs/2103.10631) [cs.IR]. URL: <https://arxiv.org/abs/2103.10631>.
- [2] Jonathan Frankle and Michael Carbin. “The Lottery Ticket Hypothesis: Training Pruned Neural Networks”. In: *CoRR* abs/1803.03635 (2018). arXiv: [1803.03635](https://arxiv.org/abs/1803.03635). URL: <https://arxiv.org/abs/1803.03635>.
- [3] Christopher Brix, Parnia Bahar, and Hermann Ney. “Successfully Applying the Stabilized Lottery Ticket Hypothesis to the Transformer Architecture”. In: *CoRR* abs/2005.03454 (2020). arXiv: [2005.03454](https://arxiv.org/abs/2005.03454). URL: <https://arxiv.org/abs/2005.03454>.
- [4] Lei Li et al. “Multimodal ArXiv: A Dataset for Improving Scientific Comprehension of Large Vision-Language Models”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 14369–14387. DOI: [10.18653/v1/2024.acl-long.775](https://doi.org/10.18653/v1/2024.acl-long.775). URL: <https://aclanthology.org/2024.acl-long.775>.

Appendix

A. Python Libraries

```
numpy==2.4.4
matplotlib==3.10.9
torch==2.11.0
datasets==4.8.5
transformers==5.8.0
evaluate[template]==0.4.6
safetensors==0.7.0
tqdm==4.67.3
huggingface_hub==1.14.0
accelerate==1.13.0
```

B. Result Tables

Tables [2](#) and [3](#) show the results for the same row of the dataset, grouped by pre-training status. The first row for both tables details the baseline (unpruned) model.

Table 2: Row 1 Results (Pre-trained before Pruning).

Trained	Pruning Percent- age	Perplexity	ROGUE-L F1 Score (%)	TTFT (ms)	Run Time (ms)	Output Tokens	Time per Output Token (ms)
	0%	32.0966	100.0000%	126.417	1,032.684	629	1.642
True	2.5%	936,589.1582	100.0000%	9.980	355.847	629	0.566
True	5.0%	1,643,765.1638	100.0000%	9.120	352.257	629	0.560
True	7.5%	1,544,174.4671	100.0000%	9.550	352.986	629	0.561
True	10.0%	1,862,629.9526	100.0000%	9.598	356.386	629	0.567
True	12.5%	2,884,897.7057	100.0000%	9.789	353.983	629	0.563
True	15.0%	18,811,896.1195	100.0000%	13.837	361.342	629	0.574
True	17.5%	24,154,952.7536	100.0000%	9.542	353.200	629	0.562
True	20.0%	10,069,282.3901	100.0000%	9.551	356.737	629	0.567
True	22.5%	7,366,844.3689	100.0000%	9.720	354.719	629	0.564
True	25.0%	18,811,896.1195	100.0000%	9.646	357.248	629	0.568
True	27.5%	12,929,214.5167	100.0000%	9.575	355.126	629	0.565
True	30.0%	18,811,896.1195	100.0000%	9.527	357.990	629	0.569
True	32.5%	5,063,153.1532	100.0000%	9.647	360.581	629	0.573
True	35.0%	18,811,896.1195	100.0000%	9.552	361.853	629	0.575
True	37.5%	39,824,784.3976	100.0000%	9.600	352.704	629	0.561
True	40.0%	14,650,719.4290	100.0000%	9.552	357.954	629	0.569
True	42.5%	7,366,844.3689	100.0000%	9.543	356.454	629	0.567
True	45.0%	11,409,991.7638	100.0000%	9.656	356.081	629	0.566
True	47.5%	8,886,110.5205	100.0000%	9.688	353.396	629	0.562
True	50.0%	11,409,991.7638	100.0000%	9.573	356.522	629	0.567

Table 3: Row 1 Results (No pre-training before pruning).

Trained	Pruning Percent- age	Perplexity	ROGUE-L F1 Score (%)	TTFT (ms)	Run Time (ms)	Output Tokens	Time per Output Token (ms)
	0%	32.0966	100.0000%	126.417	1,032.684	629	1.642
False	2.5%	1,061,294.5558	100.0000%	9.063	341.632	629	0.543
False	5.0%	2,391,664.2010	100.0000%	9.097	359.628	629	0.572
False	7.5%	1,544,174.4671	100.0000%	10.317	365.822	629	0.582
False	10.0%	1,749,778.9086	100.0000%	9.517	353.694	629	0.562
False	12.5%	2,710,110.5896	100.0000%	9.571	356.579	629	0.567
False	15.0%	31,015,573.2745	100.0000%	9.657	352.827	629	0.561
False	17.5%	31,015,573.2745	100.0000%	10.324	364.899	629	0.580
False	20.0%	10,069,282.3901	100.0000%	9.647	358.069	629	0.569
False	22.5%	8,347,728.3006	100.0000%	9.611	352.545	629	0.560
False	25.0%	24,154,952.7536	100.0000%	9.595	361.021	629	0.574
False	27.5%	8,886,110.5205	100.0000%	9.588	361.088	629	0.574
False	30.0%	16,601,440.0572	100.0000%	10.300	358.363	629	0.570
False	32.5%	6,501,217.3374	100.0000%	9.604	361.307	629	0.574
False	35.0%	14,650,719.4290	100.0000%	9.801	354.908	629	0.564
False	37.5%	45,127,392.8338	100.0000%	9.600	353.682	629	0.562
False	40.0%	21,316,670.9871	100.0000%	9.571	362.677	629	0.577
False	42.5%	8,347,728.3006	100.0000%	9.571	356.886	629	0.567
False	45.0%	8,886,110.5205	100.0000%	10.065	355.949	629	0.566
False	47.5%	11,409,991.7638	100.0000%	9.689	360.981	629	0.574
False	50.0%	8,347,728.3006	100.0000%	9.651	361.988	629	0.575

C. Results Figures

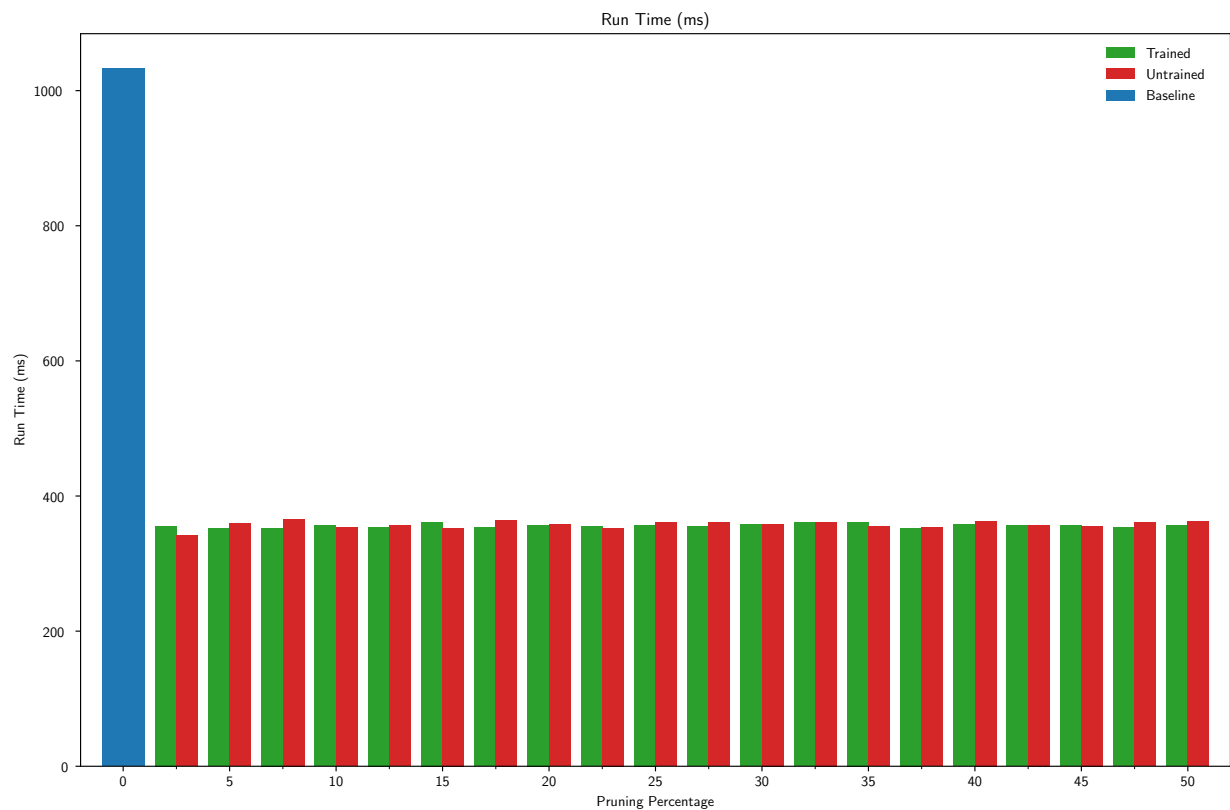


Figure 1: The total run time for each model.

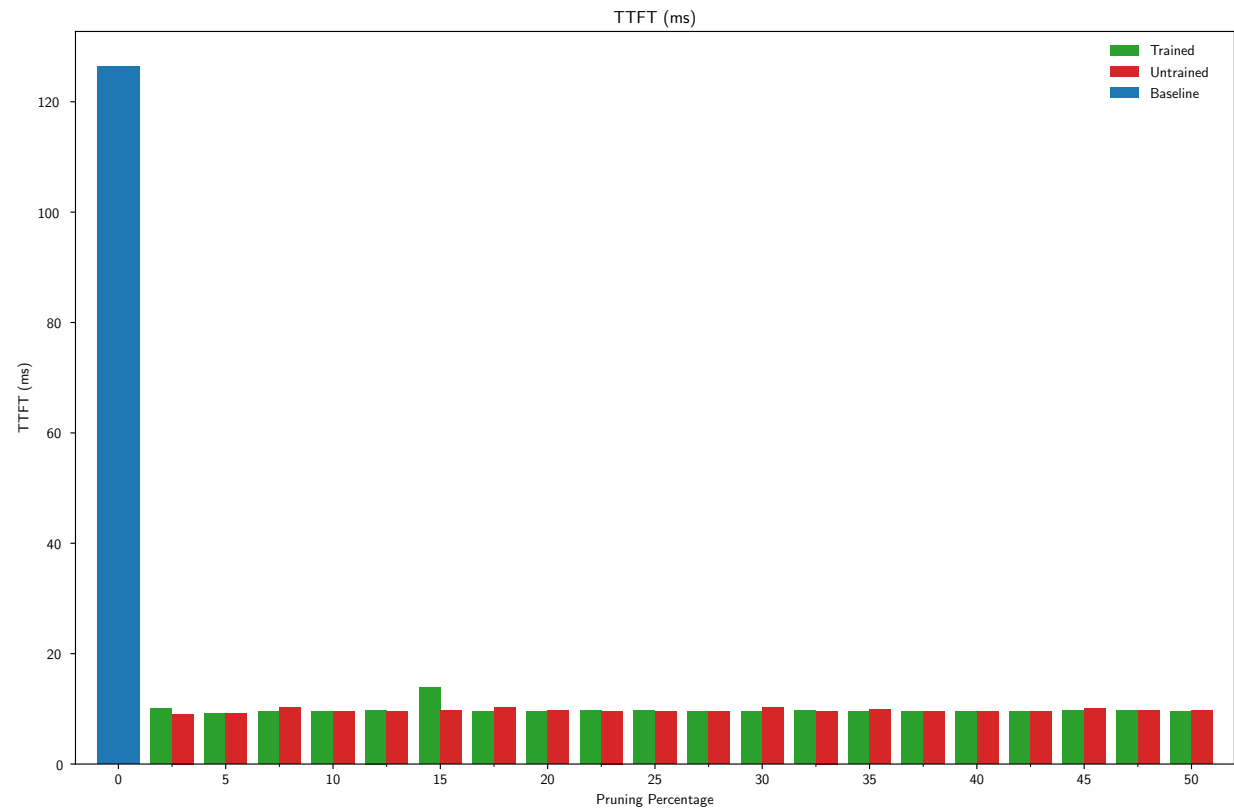


Figure 2: The amount of time it took the model to generate its first response token (Time to First Token).

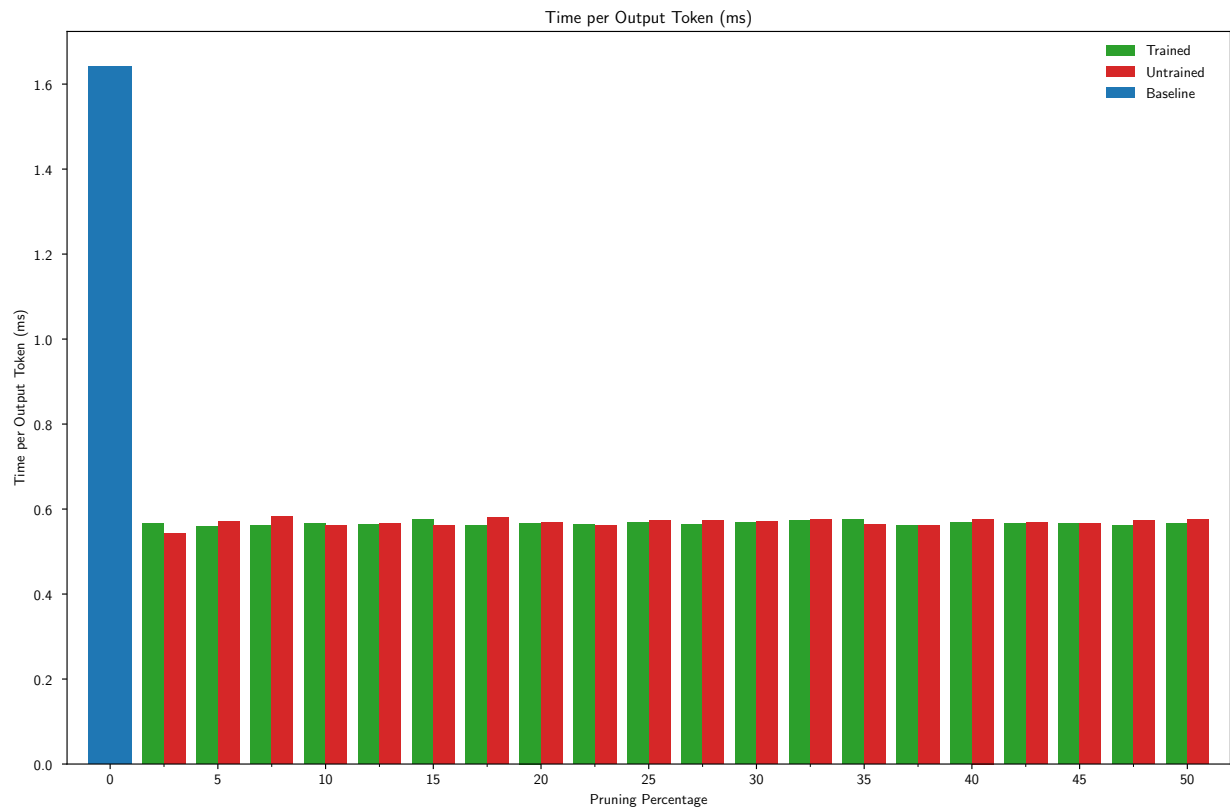


Figure 3: The average amount of time it took to generate each token in the response.

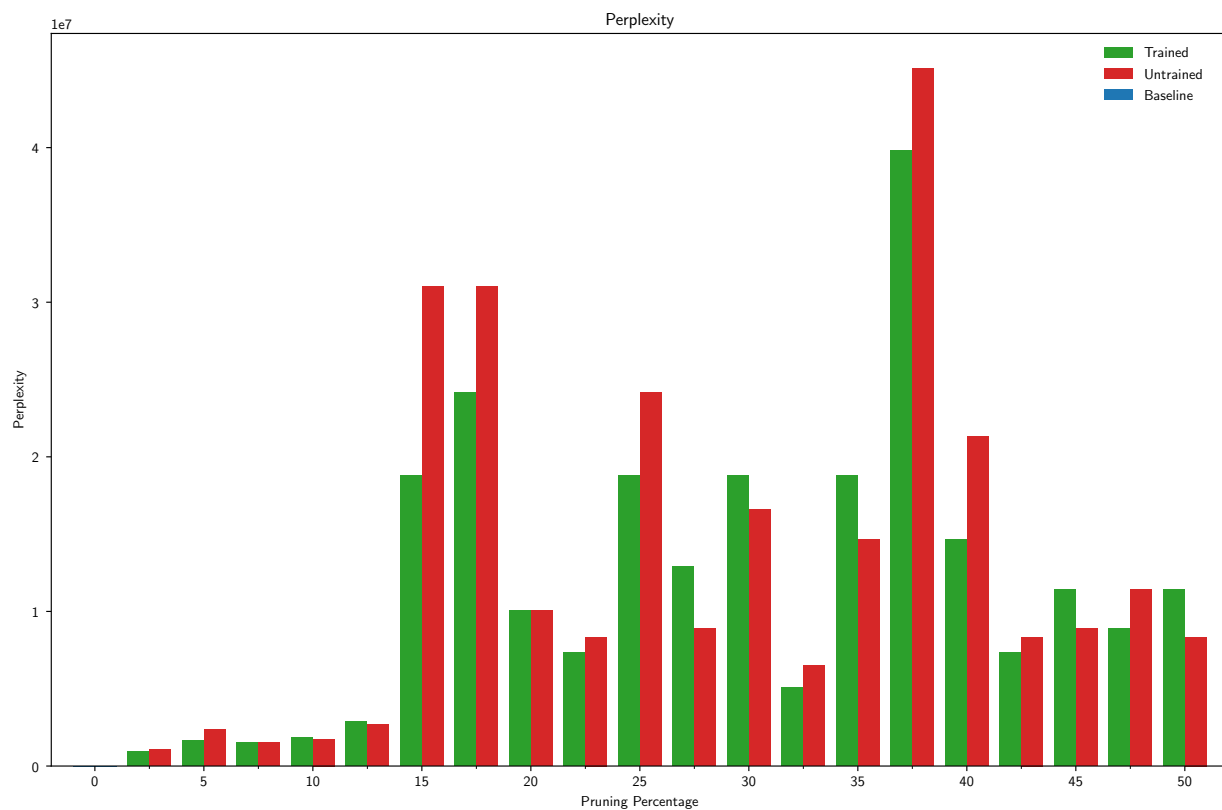


Figure 4: The Perplexity of the first row of CNN Daily Mail. The Perplexity of the Trained models is so high that the perplexity of the baseline and untrained models are too low to be seen.

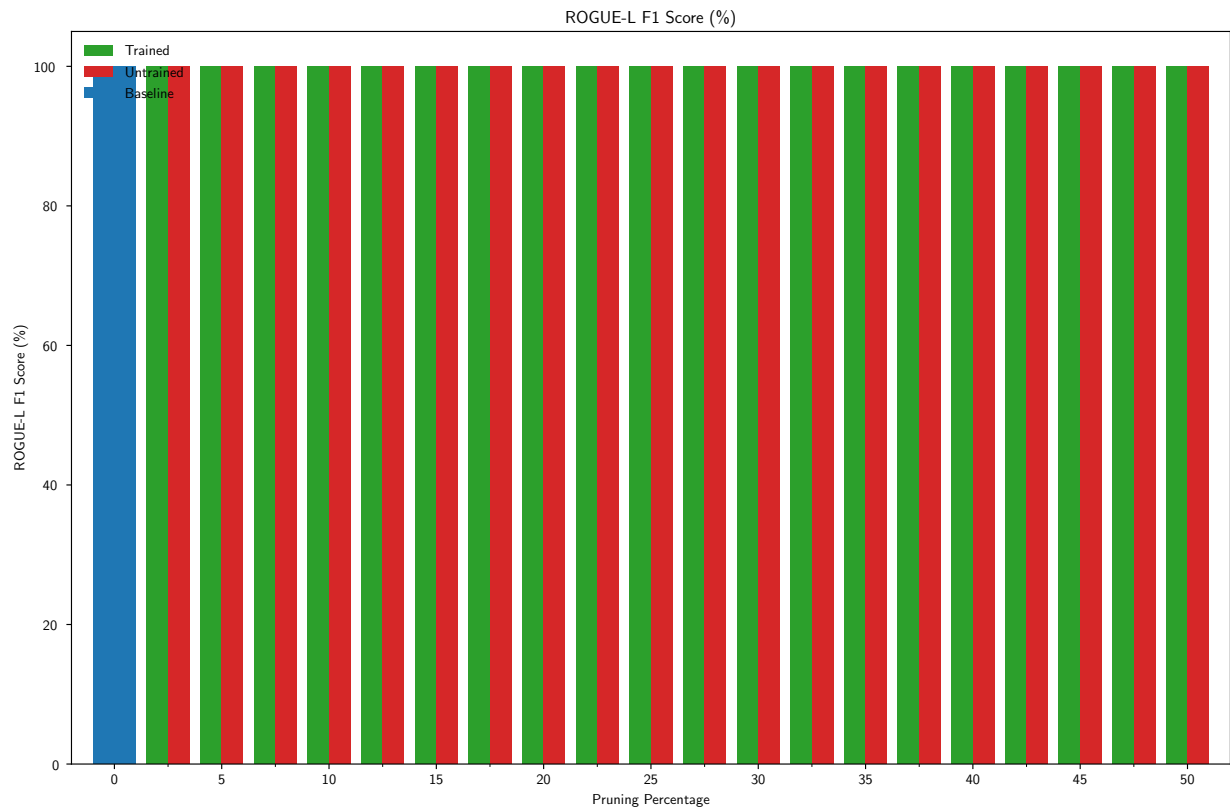


Figure 5: The ROGUE F1 Score for the Longest Common Subsequence.

D. Ablation Figures

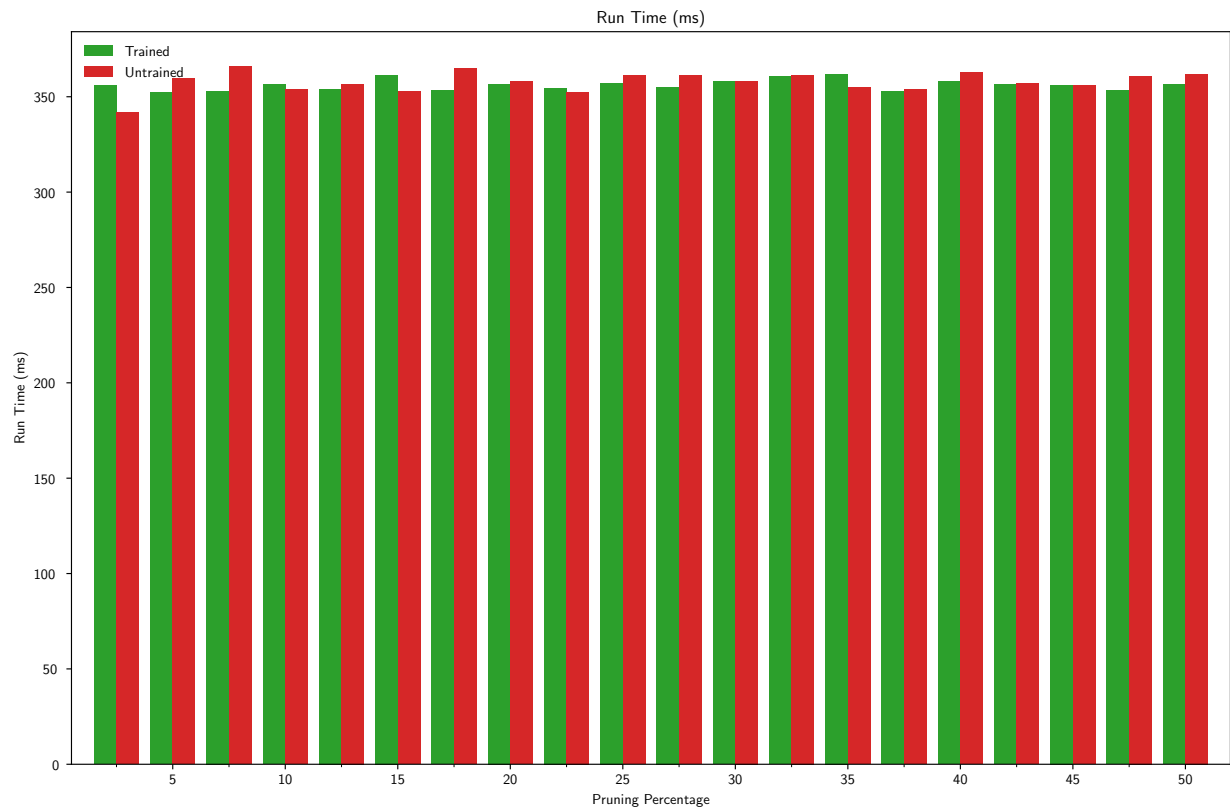


Figure 6: The total run time for each model (without the baseline).

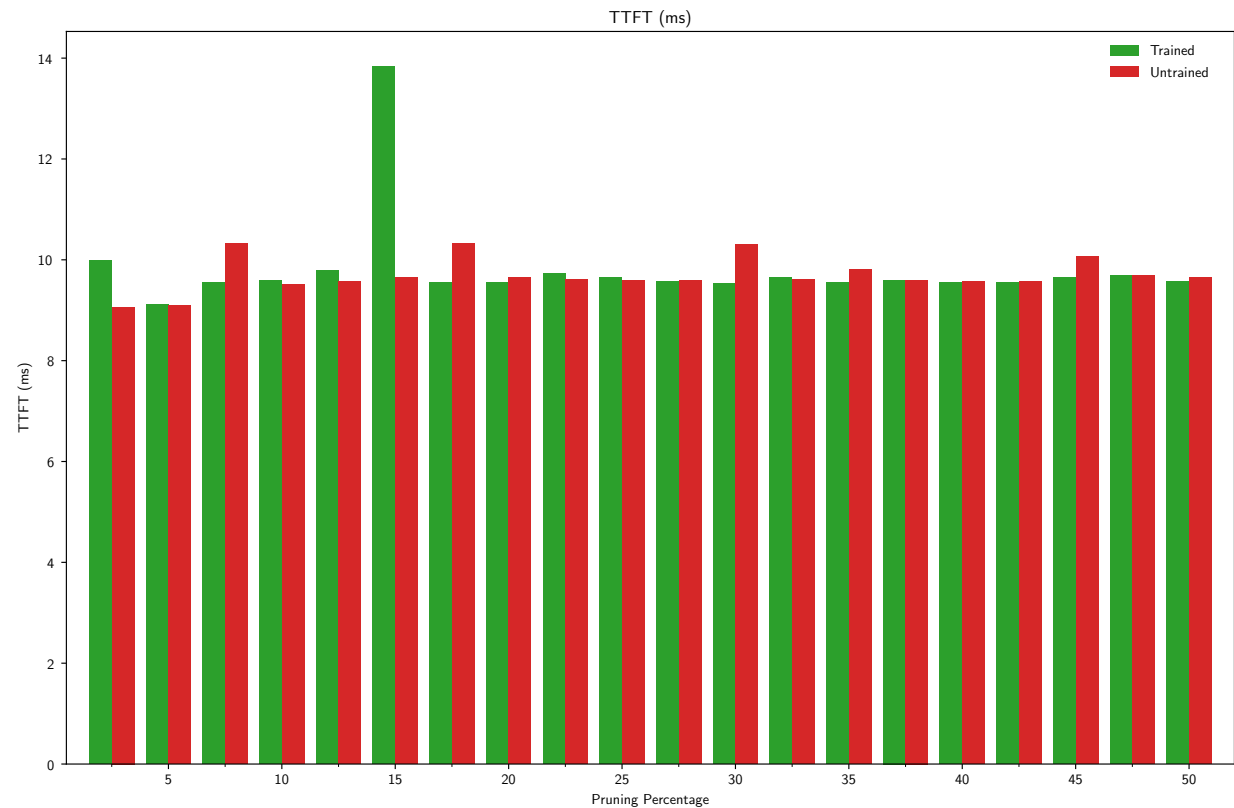


Figure 7: The amount of time it took the model to generate its first response token (Time to First Token) (without the baseline).

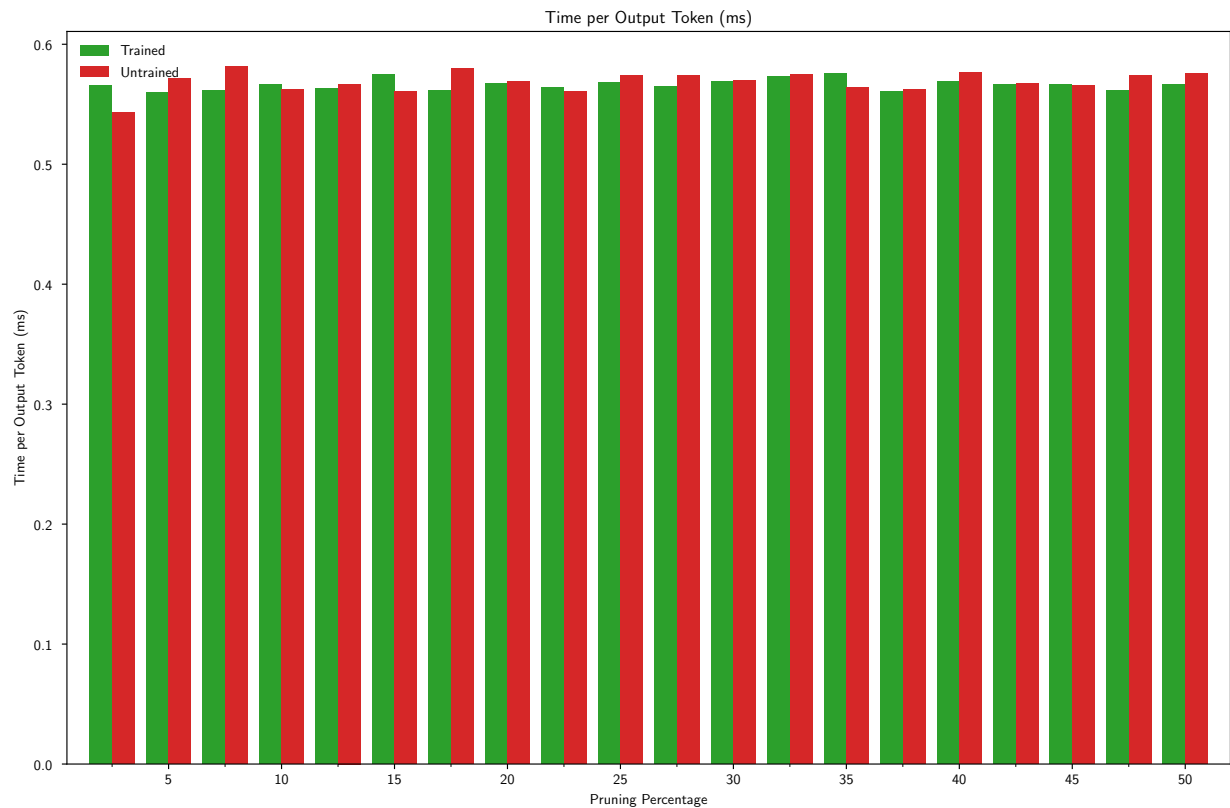


Figure 8: The average amount of time it took to generate each token in the response (without the baseline).

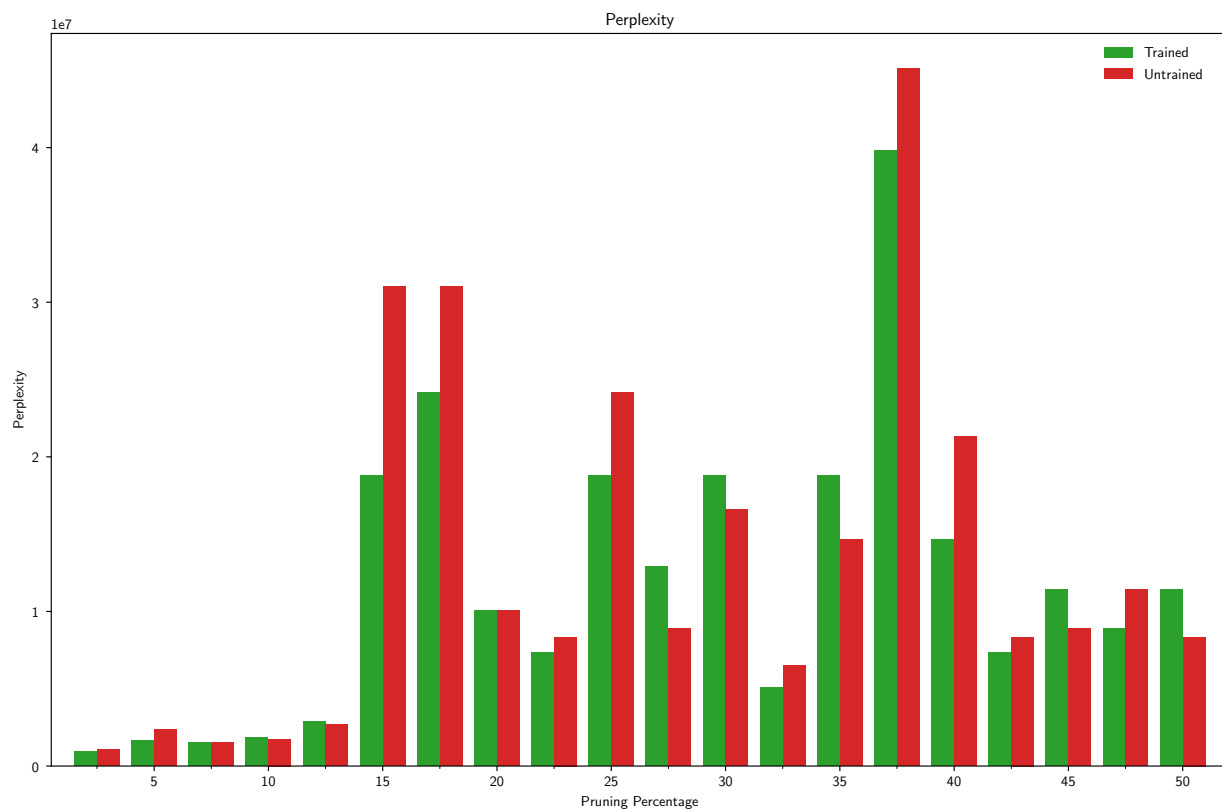


Figure 9: The Perplexity of the first row of CNN Daily Mail. The Perplexity of the Trained models is so high that the perplexity untrained models are too low to be seen.

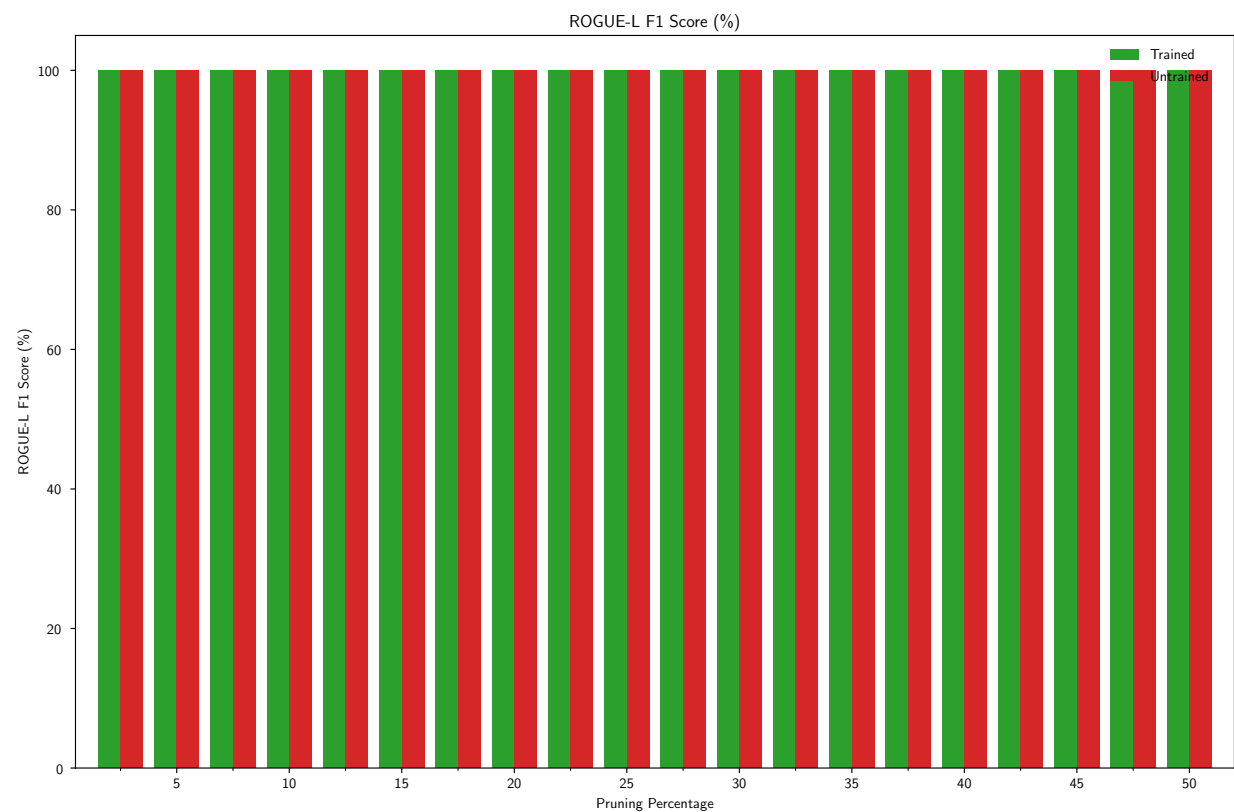


Figure 10: The ROGUE F1 Score for the Longest Common Subsequence (without the base-line).